

Track B — How to Run Guide

This guide provides step-by-step instructions for running Track B (Multi-Task FGTP Head) training, evaluation, and utilities.

Prerequisites

1. Activate the Virtual Environment

All commands must be run from the repository root with the virtual environment activated:

```
cd D:\Thesis\Ego4d-LiteSTA
.\local_extraction\.venv\Scripts\Activate.ps1
```

2. Verify Installation

Test that imports work correctly:

```
python -c "from local_extraction.trackB import trackB_train_loader;
print('OK')"
python -c "from local_extraction.trackB import trackB_eval; print('OK')"
```

3. Required Data

Ensure you have:

- **Extracted frames:** `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg`
- **Track A Stage B manifests:** `local_extraction/runs/Track_A/trackA_stageB_*/head_train.jsonl` and `head_val.jsonl`
- **Pre-extracted tokens (optional):** `local_extraction/v2/omnivore_video_swinl_fp16/` (ResNet18) or `videomae_trackB_tokens_scratch/tokens/` (VideoMAE)

Training

Quick Demo (Verify Setup)

Run a quick demo to verify everything works:

```
python local_extraction/trackB/trackB_train_loader.py --demo --epochs 1
```

Expected output:

- Progress bars for dataset parsing
- Training steps for 1 epoch
- Demo completes without errors

Full Training with ResNet18 (Default)

Train Track B with the default ResNet18 backbone:

```
python local_extraction/trackB/trackB_train_loader.py
```

Uses `configs/trackB.yaml` by default (20 epochs, batch_size=8, lr=0.001).

Training with Specific Config

Use a preset configuration:

```
# ResNet18 baseline
python local_extraction/trackB/trackB_train_loader.py --config
trackB_resnet18_baseline

# VideoMAE backbone
python local_extraction/trackB/trackB_train_loader.py --config
trackB_videomae_ego
```

Training with CLI Overrides

Override specific parameters:

```
# Custom epochs and learning rate
python local_extraction/trackB/trackB_train_loader.py --epochs 30 --lr 0.0005

# Custom batch size
python local_extraction/trackB/trackB_train_loader.py --batch_size 16

# Full custom example
python local_extraction/trackB/trackB_train_loader.py --config trackB --epochs
25 --batch_size 4 --lr 0.0003
```

Training Outputs

After training completes, outputs are saved to:

```
local_extraction/runs/Track_B/  
├── checkpoints/  
│   ├── trackB_best.pt                # Best checkpoint (highest mAP)  
│   ├── trackB_best_mAP_0.XXXX_*.pt  # Best with timestamp  
│   ├── trackB_final_*.pt            # Final epoch checkpoint  
│   └── trackB_epoch_XX_*.pt         # Per-epoch checkpoints  
├── metrics/  
│   └── metrics_train_*.json         # Training metrics  
└── plots/  
    └── trackB_train_metrics_*.png   # Training curves
```

Evaluation

Evaluate Best Checkpoint

Evaluate the best saved checkpoint on validation set:

```
python local_extraction/trackB/trackB_eval.py
```

This auto-detects `trackB_best.pt` or the latest `trackB_final_*.pt`.

Evaluate Specific Checkpoint

Evaluate a specific checkpoint:

```
python local_extraction/trackB/trackB_eval.py --checkpoint  
"local_extraction/runs/Track_B/checkpoints/trackB_best.pt"
```

Evaluate with Custom Manifest

Use a specific validation manifest:

```
python local_extraction/trackB/trackB_eval.py --val_manifest  
"local_extraction/runs/Track_A/trackA_stageB_*/head_val.jsonl"
```

TTC Mode Selection

Choose between regression and binned TTC evaluation:

```
# Regression mode (default)
python local_extraction/trackB/trackB_eval.py --ttc_mode reg

# Binned mode
python local_extraction/trackB/trackB_eval.py --ttc_mode binned
```

Evaluation Outputs

```
local_extraction/runs/Track_B/metrics/
└─ metrics_val_*.json    # Validation metrics including:
                        #   - accuracy, mAP, ttc_mae_seconds
                        #   - N_mAP, Nv_mAP, N_delta_mAP, All_mAP
                        #   - Top-5 metrics
```

Utilities

Show Training Config from Checkpoint

View the training configuration stored in a checkpoint:

```
python local_extraction/trackB/trackB_show_config.py --ckpt
"local_extraction/runs/Track_B/checkpoints/trackB_best.pt"
```

Output includes: epochs, batch_size, lr, loss weights, multi-task settings, etc.

Run Plots

Generate training metric plots:

```
python local_extraction/trackB/trackB_plots.py --runs_dir
local_extraction/runs/Track_B --out_dir local_extraction/runs/Track_B/plots
```

CLI Reference

trackB_train_loader.py

```
usage: trackB_train_loader.py [-h] [--config CONFIG] [--demo] [--epochs
EPOCHS]
                                [--batch_size BATCH_SIZE] [--lr LR]

options:
  --config CONFIG          Config name: trackB, trackB_resnet18_baseline,
trackB_videomae_ego
  --demo                  Enable demo mode (quick test with 10 steps)
  --epochs EPOCHS         Override number of epochs
  --batch_size BATCH_SIZE Override batch size
  --lr LR                 Override learning rate
```

trackB_eval.py

```
usage: trackB_eval.py [-h] [--checkpoint CHECKPOINT] [--val_manifest
VAL_MANIFEST]
                                [--stageB_run STAGEB_RUN] [--ttc_mode {reg,binned}]
                                [--hotspot {on,off}] [--clip {on,off}]

options:
  --checkpoint CHECKPOINT Checkpoint path (default: auto-detect best/final)
  --val_manifest VAL_MANIFEST
                                Explicit validation manifest path
  --stageB_run STAGEB_RUN   TrackA StageB run directory
  --ttc_mode {reg,binned}   TTC mode for N+δ evaluation
  --hotspot {on,off}        Enable/disable hotspot priors (overrides config)
  --clip {on,off}           Enable/disable CLIP re-ranking (overrides config)
```

trackB_show_config.py

```
usage: trackB_show_config.py [-h] --ckpt CKPT

options:
  --ckpt CKPT    Path to Track B checkpoint (.pt)
```

Common Workflows

1. First-Time Setup & Test

```
# Activate environment
.\local_extraction\.venv\Scripts\Activate.ps1

# Verify imports
python -c "from local_extraction.trackB import trackB_train_loader;
print('OK')"
```

```
# Run demo to verify data paths
python local_extraction/trackB/trackB_train_loader.py --demo --epochs 1
```

2. Train & Evaluate Cycle

```
# Train
python local_extraction/trackB/trackB_train_loader.py --epochs 20

# Evaluate best checkpoint
python local_extraction/trackB/trackB_eval.py

# View training config
python local_extraction/trackB/trackB_show_config.py --ckpt
"local_extraction/runs/Track_B/checkpoints/trackB_best.pt"
```

3. Ablation Study: Learning Rate

```
# LR = 0.001 (default)
python local_extraction/trackB/trackB_train_loader.py --lr 0.001 --epochs 20

# LR = 0.0005
python local_extraction/trackB/trackB_train_loader.py --lr 0.0005 --epochs 20

# LR = 0.0001
python local_extraction/trackB/trackB_train_loader.py --lr 0.0001 --epochs 20
```

4. Compare Backbones

```
# ResNet18
python local_extraction/trackB/trackB_train_loader.py --config
trackB_resnet18_baseline

# VideoMAE
```

```
python local_extraction/trackB/trackB_train_loader.py --config
trackB_videomae_ego
```

5. Quick Evaluation of Multiple Checkpoints

```
# Evaluate each checkpoint
python local_extraction/trackB/trackB_eval.py --checkpoint
"local_extraction/runs/Track_B/checkpoints/trackB_epoch_05_*.pt"
python local_extraction/trackB/trackB_eval.py --checkpoint
"local_extraction/runs/Track_B/checkpoints/trackB_epoch_10_*.pt"
python local_extraction/trackB/trackB_eval.py --checkpoint
"local_extraction/runs/Track_B/checkpoints/trackB_best.pt"
```

6. Hotspot & CLIP Ablation Study

Evaluate checkpoints with different combinations of hotspot priors and CLIP re-ranking:

```
# Define checkpoint variables
$RESNET_CKPT =
"local_extraction/runs/Track_B/checkpoints/trackB_best_mAP_0.3580_20251206_083
053.pt"
$VIDEOMAE_CKPT =
"local_extraction/runs/Track_B/checkpoints/trackB_best_mAP_0.3359_20251207_202
742.pt"

# --- ResNet18 Ablation ---
# Baseline (both OFF)
python local_extraction/trackB/trackB_eval.py --checkpoint $RESNET_CKPT --
hotspot off --clip off

# Hotspot ON only
python local_extraction/trackB/trackB_eval.py --checkpoint $RESNET_CKPT --
hotspot on --clip off

# CLIP ON only
python local_extraction/trackB/trackB_eval.py --checkpoint $RESNET_CKPT --
hotspot off --clip on

# Both ON
python local_extraction/trackB/trackB_eval.py --checkpoint $RESNET_CKPT --
hotspot on --clip on

# --- VideoMAE-Ego Ablation ---
# Baseline (both OFF)
python local_extraction/trackB/trackB_eval.py --checkpoint $VIDEOMAE_CKPT --
hotspot off --clip off
```

```
# Hotspot ON only
python local_extraction/trackB/trackB_eval.py --checkpoint $VIDEOMAE_CKPT --
hotspot on --clip off

# CLIP ON only
python local_extraction/trackB/trackB_eval.py --checkpoint $VIDEOMAE_CKPT --
hotspot off --clip on

# Both ON
python local_extraction/trackB/trackB_eval.py --checkpoint $VIDEOMAE_CKPT --
hotspot on --clip on
```

7. Best December 2025 Checkpoints Comparison

Reference checkpoints for ResNet18 vs VideoMAE-Ego comparison:

Backbone	Best Checkpoint	mAP	Accuracy	TTC MAE
ResNet18	trackB_best_mAP_0.3580_20251206_083053.pt	0.358	0.688	0.190s
VideoMAE-Ego	trackB_best_mAP_0.3359_20251207_202742.pt	0.336	0.657	0.204s

Hyperparameters for best ResNet18 run:

- lr: 0.001
- warmup_epochs: 1.0
- batch_size: 8
- label_smoothing: 0.05

Hyperparameters for best VideoMAE-Ego run:

- lr: 0.0005
- warmup_epochs: 2.0
- batch_size: 8
- video_backbone: videomae_ego
- tokens_root: videomae_trackB_tokens_scratch/tokens

Troubleshooting

Import Errors

```
# Make sure you're in repo root and venv is activated
cd D:\Thesis\Ego4d-LiteSTA
.\local_extraction\venv\Scripts\Activate.ps1
```

Missing Data Paths

If you see "No images found" or manifest errors:

- 1. Check that extracted frames exist: `ls local_extraction/v2/extracted_frames`
- 2. Check that Stage B manifests exist: `ls local_extraction/runs/Track_A/trackA_stageB_*/head_*.jsonl`

CUDA Out of Memory

Reduce batch size:

```
python local_extraction/trackB/trackB_train_loader.py --batch_size 4
```

Checkpoint Not Found

If evaluation fails to find checkpoint:

```
# List available checkpoints
ls local_extraction/runs/Track_B/checkpoints/

# Specify explicitly
python local_extraction/trackB/trackB_eval.py --checkpoint
"path/to/checkpoint.pt"
```

Configuration Files

File	Description
configs/trackB.yaml	Default configuration (ResNet18, 20 epochs)
configs/trackB_resnet18_baseline.yaml	ResNet18 baseline preset
configs/trackB_videomae_ego.yaml	VideoMAE backbone preset
CONFIGURABLE_OPTIONS.md	Full parameter reference

Key Metrics Explained

Metric	Description
accuracy	Next-active object classification accuracy
mAP	Mean Average Precision for next-active detection

Metric	Description
ttc_mae_seconds	Time-to-Contact Mean Absolute Error (seconds)
N_mAP	Noun-only mAP (N)
Nv_mAP	Noun + Verb joint mAP (N+V)
N_delta_mAP	Noun + TTC bin mAP (N+ δ)
All_mAP	Full semantic: Noun + Verb + TTC (N+V+ δ)
N_top5_mAP	Top-5 candidate Noun mAP

Expected Performance

Config	mAP	Training Time
ResNet18 (20 epochs)	~35-36%	~30-45 min
VideoMAE (20 epochs)	~33-34%	~1-2 hours

Times vary based on GPU and data loading speed.

Ablation Matrix: Hotspot & CLIP

Run all 8 combinations (2 backbones $\times$ 2 hotspot $\times$ 2 clip) and compare:

Backbone	Hotspot	CLIP	Expected Effect
ResNet18	✗	✗	Baseline
ResNet18	☑	✗	+noun/verb co-occurrence
ResNet18	✗	☑	+visual-semantic alignment
ResNet18	☑	☑	Combined boost
VideoMAE	✗	✗	Baseline (temporal)
VideoMAE	☑	✗	+noun/verb priors
VideoMAE	✗	☑	+CLIP re-ranking
VideoMAE	☑	☑	Full pipeline

Hotspot priors: Use noun-verb co-occurrence statistics from training set to boost plausible action combinations.

CLIP re-ranking: Use CLIP visual-text similarity to re-rank candidate boxes based on noun label alignment.